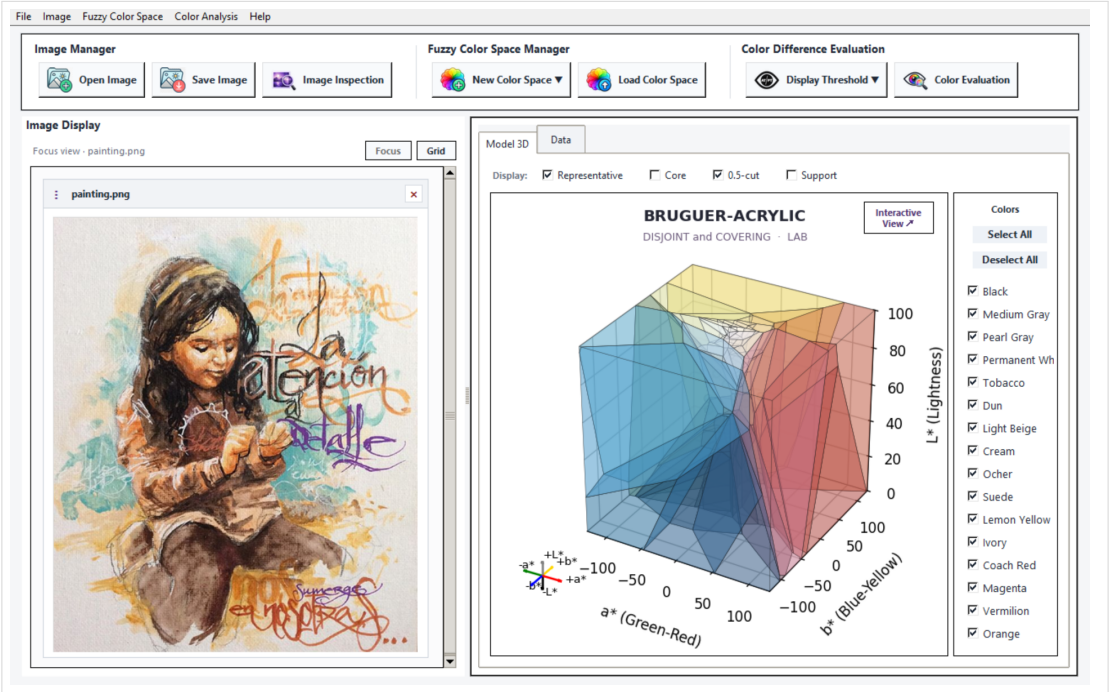
PyFCS 1.5.0

Python Fuzzy Color Software

User Manual and Technical Guide

Rafael Vázquez Conejo



PyFCS graphical interface

Version 1.5.0

https://github.com/RafaelConejo/PyFCS

https://pyfcs.com/

Open-source software for fuzzy color-space analysis

Contents

1 About This Manual **4**

2 Introduction **4**

3 System Requirements and Launching PyFCS **5**

3.1 Supported Platforms 5

3.2 Installing from Source 5

4 Theoretical Foundations **6**

4.1 Fuzzy Colors 6

4.2 Geometric Construction 6

4.3 Four Fuzzy Color Space Typologies 6

5 Main Interface **7**

5.1 Application Menu Bar 7

5.2 Main Toolbar 7

6 Fuzzy Color Space Manager **8**

6.1 Loading an Existing Color Space 8

7 Palette-Based Color Space Creation **9**

7.1 Color Table 10

7.2 Palette-Based Controls 10

7.3 Adding Multiple Sources 10

7.4 Adding a Custom Color 11

7.5 Duplicate Prototype Validation 11

8 Image-Based Color Space Creation **11**

8.1 Image Selection and Navigation 12

8.2 Manual Extraction 12

8.3 LAB Filters 13

8.4 Automatic Detection 14

8.5 Selected Colors 14

9 Color Space Configuration **15**

9.1 Choosing RGB or CIELAB Geometry 15

9.2 Selecting a Typology 16

9.3 Core, Support, and Reference Bounds 16

9.4 Negative Prototypes for Non-Disjoint Spaces 16

9.5 Auxiliary Sampling for Non-Covering Spaces . . . 17
9.6 Creating and Saving the Model . . . 17

10 Model 3D **17**
10.1 Display Options . . . 19
10.2 Interactive Figure . . . 19

11 Data Tab **20**

12 Changing Between RGB and CIELAB **21**

13 Image Manager and Image Display **22**
13.1 Opening Images . . . 22
13.2 Grid, Manual Layout, and Focus . . . 23
13.2.1 Grid . . . 23
13.2.2 Manual Layout . . . 23
13.2.3 Focus . . . 23
13.3 Image Card Controls . . . 24
13.4 Per-Image Menu . . . 25
13.5 Pixel and ROI Sampling in the Main Display . . . 25

14 Color Mapping **26**
14.1 Single-Prototype Color Mapping . . . 26
14.2 Color Mapping All . . . 26

15 Saving Images **28**
15.1 Saving Directly from an Image Card . . . 28
15.2 Global Save Image Selector . . . 28
15.3 Mapping-All Legend Export . . . 29

16 Image Inspection **29**
16.1 Image Source Controls . . . 30
16.2 Inspection Modes . . . 30
16.3 Selected Sample . . . 31
16.4 Membership Degree . . . 31
16.5 Comparison Settings . . . 32
16.6 Prototype Details . . . 32
16.7 Pinned Samples . . . 33

17 Color Evaluation **33**

17.1 Color Space Data . . . 34
17.2 Comparison Sources . . . 34
17.3 Metrics and Thresholds . . . 34
17.4 Closest Prototypes and Membership Analysis . . . 35
17.5 Graphical Analysis . . . 35

18 Display Threshold **35**

19 Recommended Workflows **35**
19.1 Workflow A: Build an FCS from a Known Palette . . . 35
19.2 Workflow B: Build an FCS from an Image Manually . . . 36
19.3 Workflow C: Automatic Image Palette Extraction . . . 36
19.4 Workflow D: Map an Image with an Existing FCS . . . 37
19.5 Workflow E: Inspect a Sample in Detail . . . 37
19.6 Workflow F: Rebuild an Existing FCS in Another Model . . . 37

20 File Formats and Output Behavior **37**
20.1 Color-Space Files . . . 37
20.2 Compatibility and Granularity . . . 38
20.3 Image Outputs . . . 38

21 Mouse and Keyboard Reference **38**
21.1 Global Keyboard Controls . . . 38
21.2 Image Interaction Reference . . . 38

22 Good Practice **39**

23 Troubleshooting **39**
23.1 A Mapping Option Is Disabled . . . 39
23.2 An Image Card Overlaps Another . . . 39
23.3 Focus Cannot Be Pressed Again . . . 40
23.4 Right-Drag Does Not Move the Image . . . 40
23.5 The Wrong Pixel Appears to Be Selected . . . 40
23.6 A New FCS Cannot Be Created . . . 40
23.7 Change Color Space Does Not Complete . . . 40
23.8 Loading or Processing Appears Busy . . . 40
23.9 Mapping-All Legend Is Missing on Export . . . 40

24 Contact and Support **41**

1 About This Manual

This manual describes the graphical version of **PyFCS** as available in version 1.5.0. Its purpose is not only to list the available features, but to explain the intended workflow of the application, the role of each control, and the relationship between image analysis, fuzzy color-space construction, visualization, color mapping, and color-difference evaluation.

> **Important.** The user interface is designed for Windows, Linux, and macOS. Small visual differences in window decorations, menu rendering, fonts, scrollbars, mouse buttons, and file dialogs are expected because Tkinter uses platform-dependent widgets.

2 Introduction

PyFCS is a scientific desktop application for creating, visualizing, analyzing, and applying *Fuzzy Color Spaces* (FCS). Rather than assigning a rigid color label, an FCS models linguistic colors through membership degrees, allowing gradual transitions and potentially overlapping categories.

Version 1.5.0 introduces native geometric construction in both **RGB** and **CIELAB**, supports the **four fuzzy color-space typologies**, and provides configurable cores, supports, reference bounds, and negative prototypes. The `Change Color Space` action allows an existing model to be rebuilt in the other coordinate system. The same `.fcs` format is retained for legacy, extended, and future granular data.

Version 1.5.0 combines several workflows in a single desktop environment:

- creation of RGB- or CIELAB-based fuzzy color spaces from palettes, existing models, custom colors, or image samples;
- selection of disjoint/non-disjoint and covering/non-covering typologies, geometric scaling, bounds, and negative prototypes;
- interactive 3D visualization of representatives, cores, 0.5-cuts, and supports in the active geometric model;
- reconstruction of an FCS between RGB and CIELAB;
- editing of the color-space data;
- loading and simultaneous management of multiple images;
- pixel and region-of-interest (ROI) sampling;
- single-prototype color mapping and complete fuzzy color mapping;
- detailed **Image Inspection** with fuzzy memberships and color-difference analysis;
- configurable color-difference evaluation and threshold analysis;
- export of processed images and Mapping-All legends;
- responsive loading, geometric processing, and progress feedback.

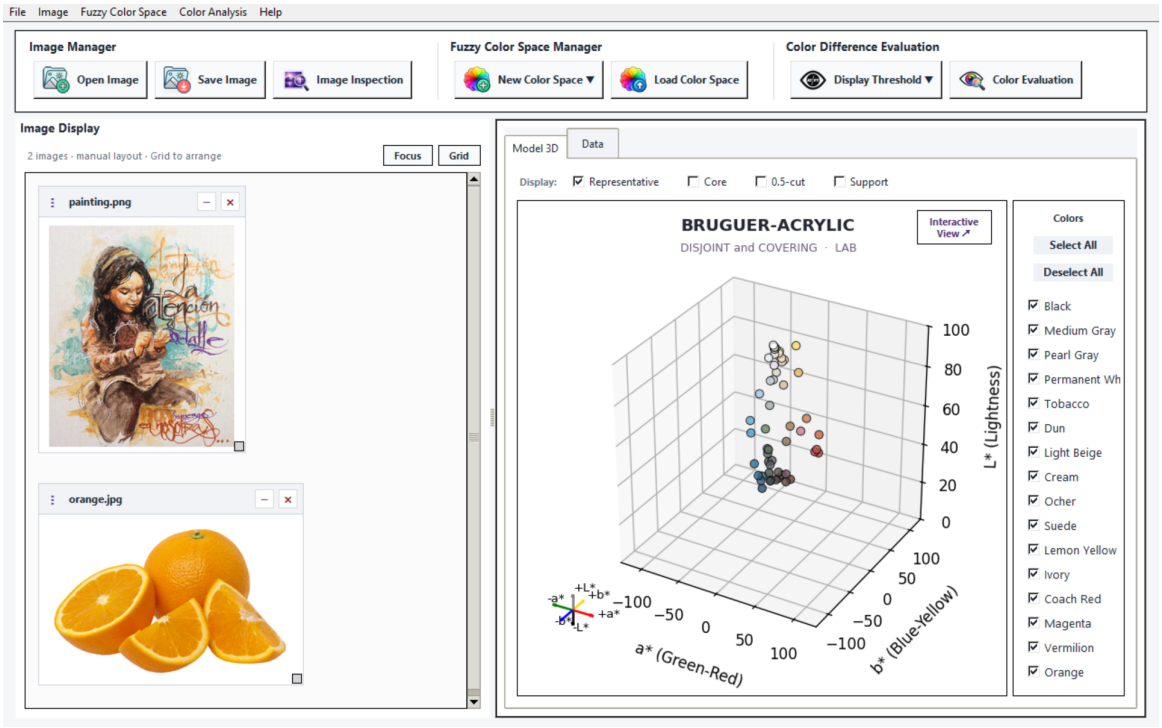


Figure 1: Main PyFCS 1.5.0 interface.

3 System Requirements and Launching PyFCS

3.1 Supported Platforms

The source version of **PyFCS** is intended to run on:

- Microsoft Windows;
- Linux distributions with Python and Tk support;
- macOS with a Python/Tk installation compatible with the project dependencies.

Python 3.9 or later is required; Python 3.10 is recommended for a conservative, well-tested scientific Python environment.

3.2 Installing from Source

The project is available at:

https://github.com/RafaelConejo/PyFCS

When using the source distribution:

1. download or clone the repository;
2. create or activate the Python environment to be used with PyFCS;
3. install the dependencies specified by the repository;
4. launch the current GUI entry point:

```
python Source/interface/mainInterface.py
```

For packaged releases, use the executable or launcher supplied for the target platform. Release-specific installation instructions should take precedence over generic source-installation instructions.

4 Theoretical Foundations

4.1 Fuzzy Colors

A fuzzy color represents a color category with gradual rather than crisp boundaries. Given a color space C (RGB or CIELAB in PyFCS), a fuzzy color $\widetilde{C}$ can be described by a membership function

$$\mu_{\widetilde{C}} : C \to [0, 1].$$

A value near 1 indicates strong membership in the category, while a value near 0 indicates weak or absent membership.

A fuzzy color space is a collection

$$\widetilde{\Gamma} = \{\widetilde{C}_1, \ldots, \widetilde{C}_m\}$$

defined over a crisp color space Γ. Each category has a positive representative expressed in the *active geometric model*: RGB or CIELAB. The currently displayed RGB/HEX swatches are color previews, not an indication of which geometry was used.

4.2 Geometric Construction

The PyFCS model follows a prototype- and Voronoi-based construction. In version 1.5.0, each linguistic color uses one positive prototype together with the negatives required by the chosen construction. Voronoi regions are used to derive fuzzy-color volumes, with reference-domain boundaries and optional auxiliary negatives accounted for during construction.

The principal geometric levels shown by the software are:

- **Representative**: the positive prototype in the selected RGB or CIELAB coordinate system;
- **Core**: the region of maximum membership;
- 0.5**-cut**: the central Voronoi-derived region;
- **Support**: the outer region in which membership remains greater than zero.

Membership degrees between these regions are obtained by interpolation. Whether categories overlap and whether they cover the entire reference domain depends on the selected typology. The geometric levels describe fuzzy membership, whereas color-difference metrics provide an additional comparison in the color-evaluation tools.

4.3 Four Fuzzy Color Space Typologies

The `Disjoint` and `Covering` options in the creation dialog provide four configurations:

Typology	Interpretation
Disjoint and Covering (Partition)	Categories do not overlap and the reference domain is covered.
Disjoint and Non-Covering	Categories remain non-overlapping, but regions of the domain may be left outside all supports.
Non-Disjoint and Covering	Supports can overlap and the reference domain is covered.
Non-Disjoint and Non-Covering	Supports can overlap and some domain regions may remain uncovered.

The *Disjoint* option controls the permitted overlap; the *Covering* option controls whether all points in the reference domain must be covered. These are geometric properties and should not be confused with the choice of RGB versus CIELAB coordinates.

5 Main Interface

The main interface is divided into four functional areas:

1. the application menu bar;
2. the main toolbar;
3. the **Image Display** workspace on the left;
4. the **Model 3D / Data** notebook on the right.

5.1 Application Menu Bar

The top menu contains the following categories:

Menu	Purpose
`File`	Contains the application exit action.
`Image`	Open and save images, open Image Inspection, and close all images.
`Fuzzy Color Space`	Create an RGB/CIELAB fuzzy color space using Palette-Based or Image-Based Creation, or load an existing `.fcs`/`.cns` source.
`Color Analysis`	Open the Color Evaluation workspace.
`Help`	Open the About dialog and version information.

On Windows and Linux, the menu categories can also be accessed through the mnemonics **Alt+F**, **Alt+I**, **Alt+C**, **Alt+A**, and **Alt+H**. macOS uses the native Tk/Aqua menu behavior instead of forcing Option-key bindings.

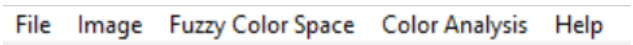


Figure 2: Application menu bar.

5.2 Main Toolbar

The toolbar groups the most frequently used actions into three sections.

Section	Button	Function
Image Manager	`Open Image`	Loads an image into the main Image Display workspace. Multiple images may be open simultaneously.
Image Manager	`Save Image`	Opens the visual image selector and exports the current full image view. If the selected image is in Color Mapping All mode, its legend is exported as a separate `_legend.png` file.
Image Manager	`Image Inspection`	Opens the dedicated inspection workspace. A fuzzy color space must be loaded, but an image can be opened later from inside the inspection window.
Fuzzy Color Space Manager	`New Color Space`	Opens a dropdown with `Palette-Based Creation` and `Image-Based Creation`.
Fuzzy Color Space Manager	`Load Color Space`	Loads a legacy or extended `.fcs`, or a `.cns` source, into the main application.
Color Difference Evaluation	`Display Threshold`	Opens a dropdown for visualizing the Perceptibility Threshold (PT) or Acceptability Threshold (AT) in the active fuzzy color space.
Color Difference Evaluation	`Color Evaluation`	Opens the detailed color-difference evaluation workspace.

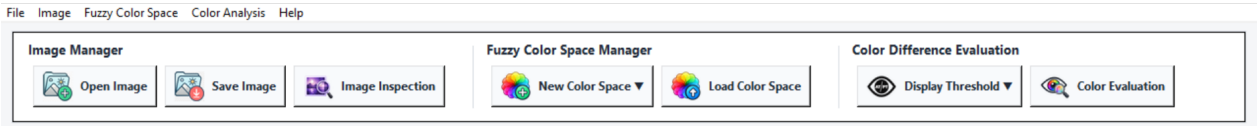


Figure 3: Main toolbar and its three functional groups.

6 Fuzzy Color Space Manager

6.1 Loading an Existing Color Space

Use `Load Color Space` to load an existing source.

Supported formats are:

- `.fcs`: a complete fuzzy color-space file, including legacy and extended PyFCS/JFCS-compatible representations;
- `.cns`: a color-name-system palette used to construct the fuzzy geometry in the application.

The application reads and prepares the new model in the background before replacing the currently active color space. The **Loading** dialog stays responsive during this operation. If loading fails, the previous valid color space remains active.

Extended `.fcs` files can preserve model coordinates, typology, reference-domain limits, scaling factors, and negative-prototype settings. Legacy files remain supported. A `.cns` is a palette source rather than a precomputed fuzzy geometry; PyFCS constructs the corresponding model when loading it.

After loading succeeds, PyFCS updates:

- the Model 3D tab;
- the color selection controls;
- the Data tab;
- the fuzzy-membership model used by Image Inspection and Color Evaluation;
- the model and typology information shown in the workspace.

If an image is already showing a mapping generated with another color space, PyFCS protects the displayed result from being silently interpreted as a mapping of the newly loaded space. New mappings are always calculated from the original image.

7 Palette-Based Color Space Creation

Palette-Based Creation is intended for building a new fuzzy color space from known reference colors. It can combine colors from the default palette, other `.cns` palettes, existing `.fcs` files, and custom user colors.

Open it through:

`New Color Space` → `Palette-Based Creation`

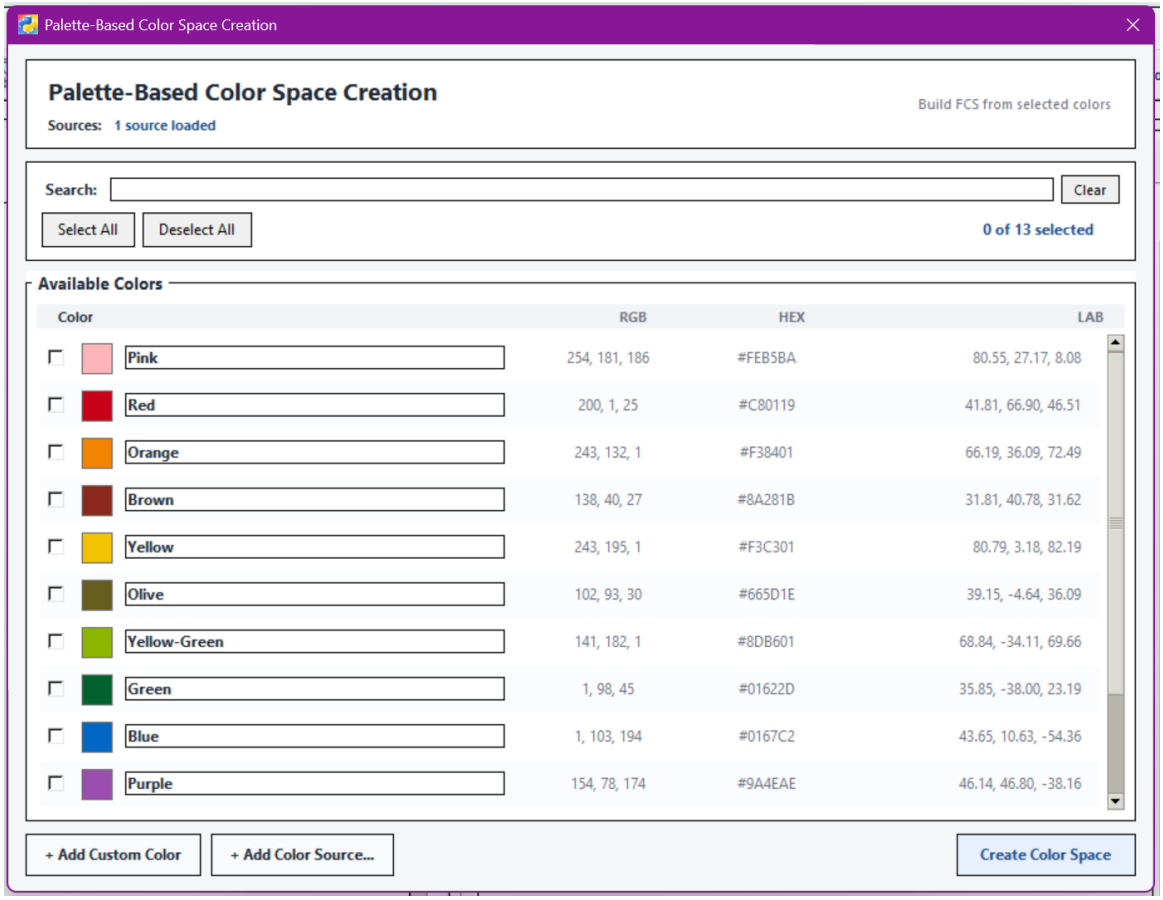


Figure 4: Palette-Based Color Space Creation workspace.

7.1 Color Table

The central table contains four information columns:

- **Color**: selection checkbox, swatch, and editable name;
- **RGB**: RGB representation;
- **HEX**: hexadecimal representation;
- **LAB**: CIELAB representative.

The names shown in this workspace are local to the new color space being constructed. Renaming an imported color does **not** modify the original CNS/FCS source file.

7.2 Palette-Based Controls

Control	Function
`Search`	Filters the visible table so that colors can be found quickly. The underlying selection state is preserved while filtering.
`Clear`	Clears the search query and restores the complete visible list.
`Select All`	Selects the available colors for inclusion in the new FCS.
`Deselect All`	Clears all current selections.
`+ Add Custom Color`	Opens the reusable color-entry dialog. A custom color can be defined through RGB, CIELAB, HEX, or the visual color picker.
`+ Add Color Source...`	Loads an additional `.cns` or `.fcs` source and appends its colors to the current workspace. Existing colors are not removed.
`Create Color Space`	Validates the selected colors, opens the **Color Space Configuration** dialog for name and geometry settings, generates the fuzzy geometry, and saves the resulting `.fcs`.

7.3 Adding Multiple Sources

Additional sources are accumulated in the same creation workspace. Loading an FCS imports its representative/positive prototype colors; it does not replace the FCS currently active in the main PyFCS interface.

Source identity is checked using a normalized file name. Consequently, sources such as `BASIC.cns` and `basic.fcs` are treated as having the same source name. If the user attempts to add a source name that has already been loaded, PyFCS asks whether it should be loaded again.

If two imported colors use the same label, the new local label is automatically differentiated, for example:

```
Red
Red (2)
Red (3)
```

The user can then edit these names directly in the table.

7.4 Adding a Custom Color

Press `+ Add Custom Color` to create a new color manually. The same reusable color-entry dialog used in other parts of PyFCS is opened.

A custom color can be defined using:

- **RGB**: enter the red, green, and blue components;
- **CIELAB**: enter the L^*, a^*, and b^* coordinates;
- **HEX**: enter a hexadecimal color value;
- the visual color picker, when available for the selected input mode.

The dialog validates the entered values and displays a preview of the resulting color. A name must also be provided before the color can be added to the Palette-Based Creation workspace.

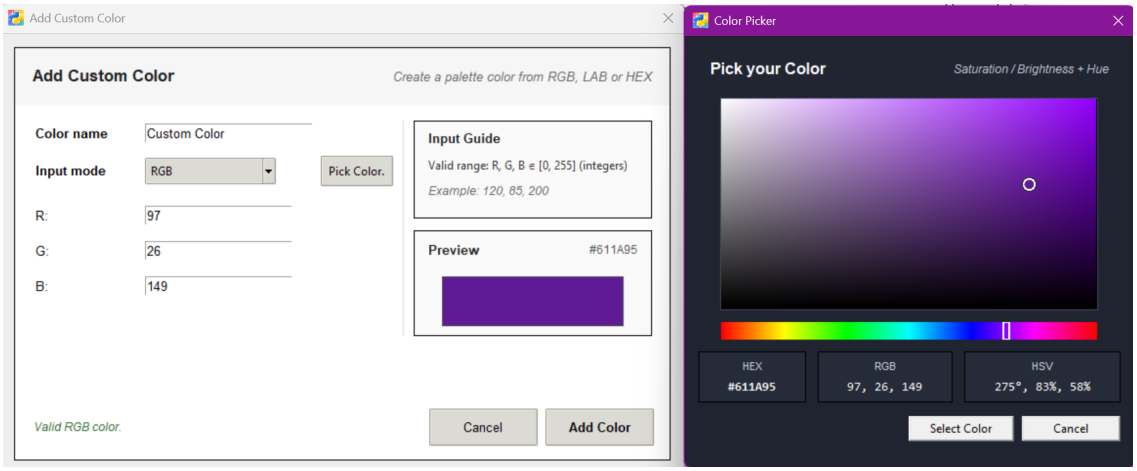


Figure 5: Adding a custom color using the reusable RGB, CIELAB, or HEX color-entry dialog.

7.5 Duplicate Prototype Validation

Different labels are allowed, but two selected colors cannot occupy the same prototype position within the geometric merge tolerance. PyFCS validates coincident representatives before constructing the Voronoi geometry. The check applies to the coordinates used by the selected model (RGB or CIELAB).

If a duplicate is detected, modify or remove one of the conflicting colors before creating the FCS.

8 Image-Based Color Space Creation

Image-Based Creation provides a single workspace for extracting representative colors from images.

Open it through:

`New Color Space` → `Image-Based Creation`

The workspace can use images already open in the main Image Display and can also open temporary local images. Images opened only in this creation workspace do not have to be added to the main application workspace.

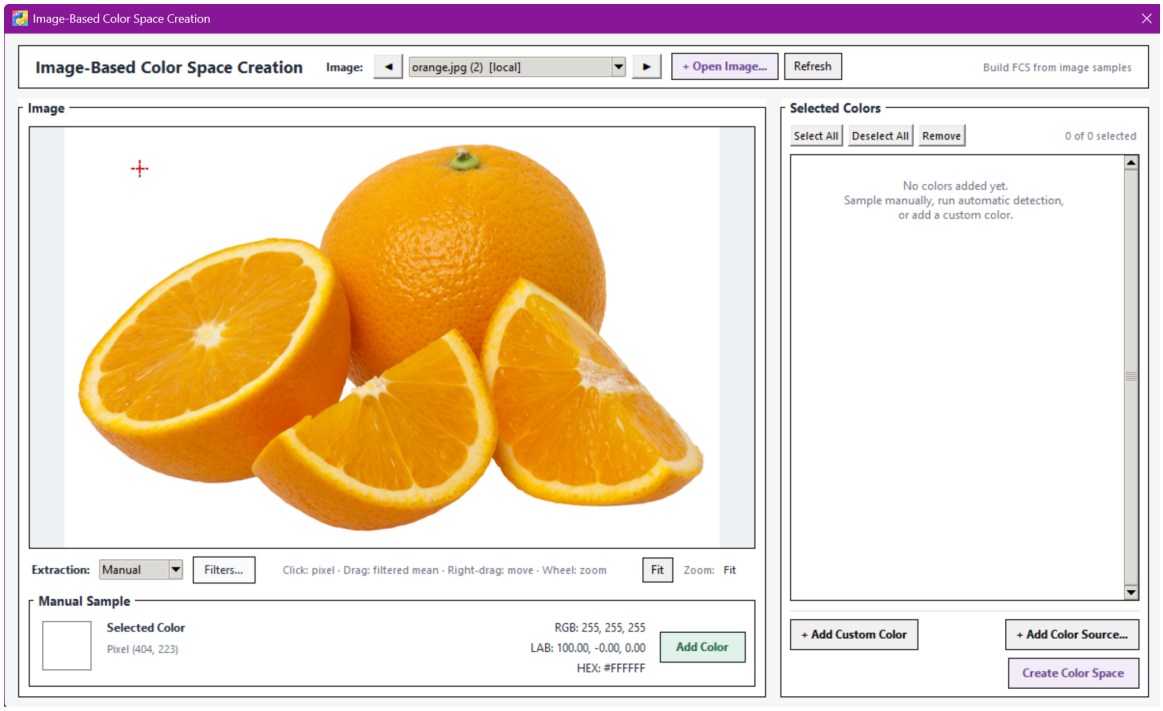


Figure 6: Image-Based Color Space Creation workspace.

8.1 Image Selection and Navigation

Control	Function
◁ / ▷	Move to the previous or next available source image.
Image selector	Select a source directly from the dropdown. Local images are marked separately from images originating in the main workspace.
`+ Open Image...`	Open a temporary image for this creation session.
`Refresh`	Refresh the list of images available from the main workspace. Images closed in the main window are removed from the selector.
`Fit`	Reset the current image viewport so that the complete image fits in the image area.
Mouse wheel	Zoom in or out around the cursor.
Right-drag / middle-drag	Move the viewport while zoomed.

Each image keeps its own viewport state, so changing to another image does not force all sources to use the same zoom and position.

8.2 Manual Extraction

Manual extraction is useful when colors must come from semantically meaningful parts of an image.

The basic interactions are:

- **click**: sample one pixel;
- **drag**: select a rectangular ROI and compute its mean color;

- **right-drag or middle-drag**: move the zoomed image;
- **mouse wheel**: zoom.

After a sample is created, PyFCS shows the extracted color and allows it to be added to the Selected Colors list.

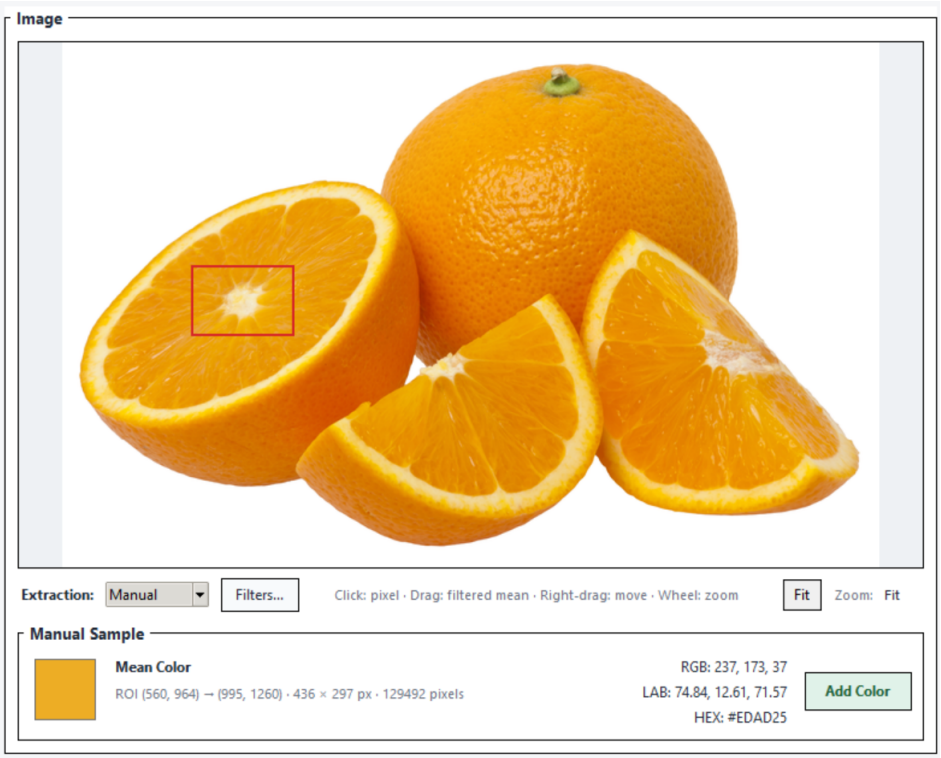


Figure 7: Manual pixel and ROI sampling in Image-Based Creation.

8.3 LAB Filters

The `Filters...` control opens optional CIELAB limits for:

- minimum and/or maximum L^*;
- minimum and/or maximum a^*;
- minimum and/or maximum b^*.

For a manually selected ROI, enabled limits are applied **pixel-by-pixel before the mean color is computed**. This is useful for excluding highlights, very dark pixels, or chromatic regions that should not contribute to the representative color.

A single clicked pixel is not altered by the ROI filtering process.

For automatic detection, the filters are applied to the detected representative colors before they are added to the Selected Colors list. This keeps the clustering process itself independent from the optional post-detection filtering.

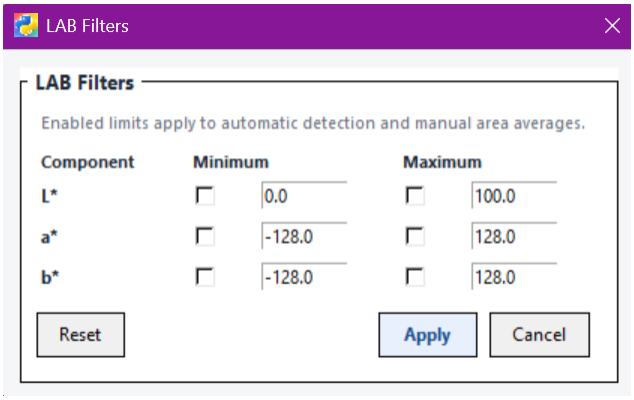


Figure 8: LAB Filters dialog used by Image-Based Creation.

8.4 Automatic Detection

Automatic extraction identifies representative colors from the selected image. The detection workflow is intended for faster palette generation when manual sampling would be unnecessarily slow.

Use `Detect and Add Colors` to run the current detection settings. The processing runs without blocking the interface; while it is active, the image area displays a detection indicator and controls that could invalidate the operation are temporarily disabled.

The resulting colors are appended to the Selected Colors list.

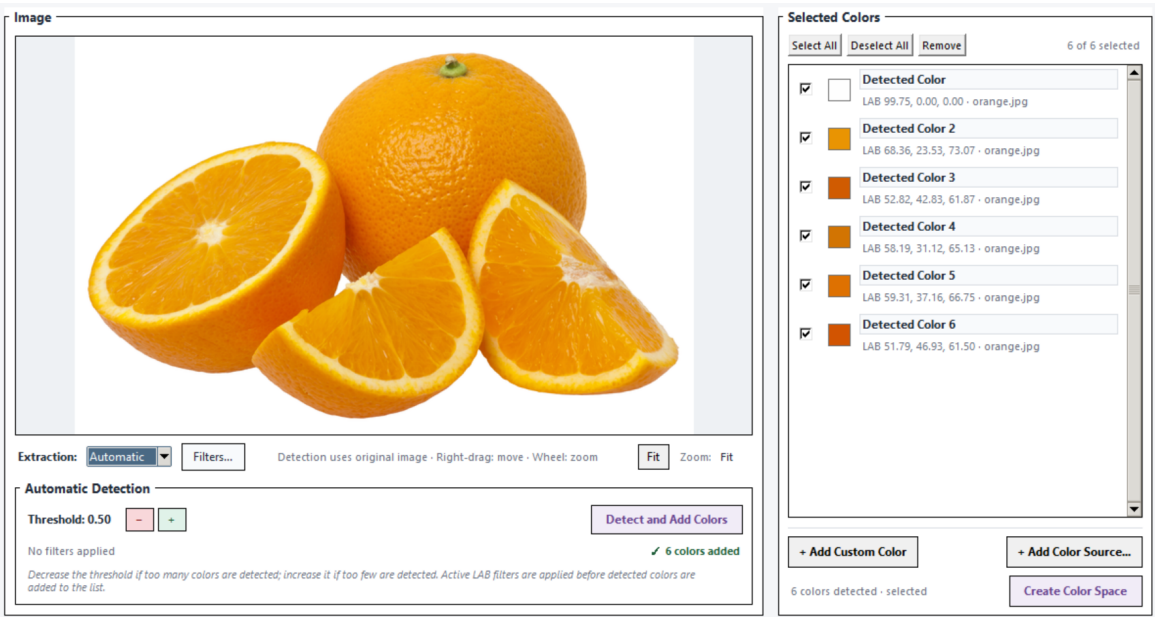


Figure 9: Automatic color detection in Image-Based Creation.

8.5 Selected Colors

The right-hand list is the temporary color set from which the FCS will be created.

Control	Function
`Select All`	Select all extracted/custom colors.
`Deselect All`	Deselect all colors.
`Remove`	Remove the selected entries from the temporary creation list.

Control	Function
`+ Add Custom Color`	Add a manually defined RGB/LAB/HEX color.
Editable color name	Rename a selected-color entry before creating the FCS.
`Create Color Space`	Validate the selected colors and open the configuration dialog before saving the new fuzzy color space.

At least two valid, non-coincident representatives are required. Both image-based and palette-based creation use the same configuration dialog described in Section 9.

9 Color Space Configuration

The final step of both **Palette-Based Creation** and **Image-Based Creation** is the `Create Color Space` dialog. Here, the selected colors are converted into a complete fuzzy geometric model. The original source palettes and images are not modified.

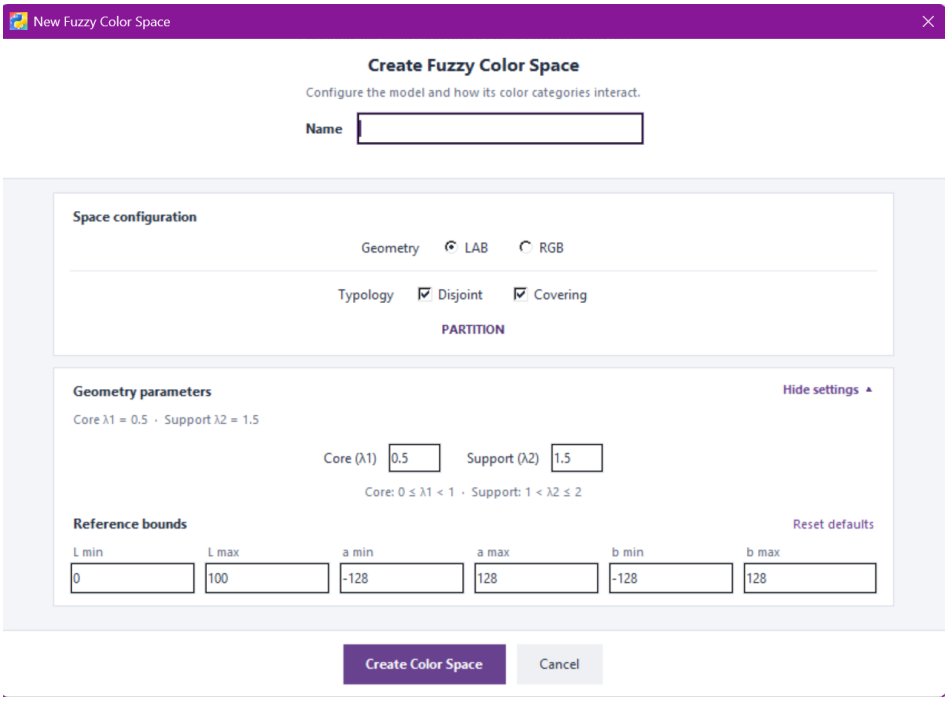


Figure 10: Color Space Configuration, including model, typology, and geometric parameters.

9.1 Choosing RGB or CIELAB Geometry

In **Space configuration**, select one of:

- `LAB`: construct the Voronoi geometry in CIELAB;
- `RGB`: construct the geometry in 8-bit RGB coordinates.

The default for a newly created FCS is **LAB**. Switching geometry also updates the reference-domain defaults and the coordinates displayed for negative-prototype selection. Distances, planes, and membership degrees are computed in the chosen geometric model; the choice is not merely a different way of labeling the axes.

9.2 Selecting a Typology

The `Disjoint` and `Covering` checkboxes set the four combinations described in the theoretical section. Both are checked by default, producing the **Partition** configuration.

For non-disjoint spaces, the dialog additionally exposes a per-category selection of negative prototypes. For non-covering spaces, it exposes an auxiliary sampling density. These controls appear only when relevant to the selected typology.

9.3 Core, Support, and Reference Bounds

Open `Edit settings` within **Geometry parameters** to edit:

- **Core** (λ_1): controls core scaling; valid values satisfy $0 \leq \lambda_1 < 1$;
- **Support** (λ_2): controls support scaling; valid values satisfy $1 < \lambda_2 \leq 2$;
- **Reference bounds**: the minimum and maximum coordinates of each axis in the selected model.

The defaults are $\lambda_1 = 0.5$ and $\lambda_2 = 1.5$. For a disjoint FCS, the additional condition $1 \leq \lambda_1 + \lambda_2 \leq 2$ applies.

The default reference bounds are $R, G, B \in [0, 255]$ for RGB, and $L^* \in [0, 100]$, $a^*, b^* \in [-128, 128]$ for LAB. Use `Reset defaults` to restore the bounds for the selected model.

9.4 Negative Prototypes for Non-Disjoint Spaces

When `Disjoint` is unchecked, a **Negative prototypes by category** panel becomes available. Choose a fuzzy-color category, then move other colors between **Available colors** and **Selected negatives**. The `Add`, `Remove`, `Add all`, and `Clear` controls edit the negatives of the currently selected category.

This allows individual categories to have different sets of negatives. Unselected colors are not automatically treated as negatives for that category; consequently, fuzzy supports may overlap.

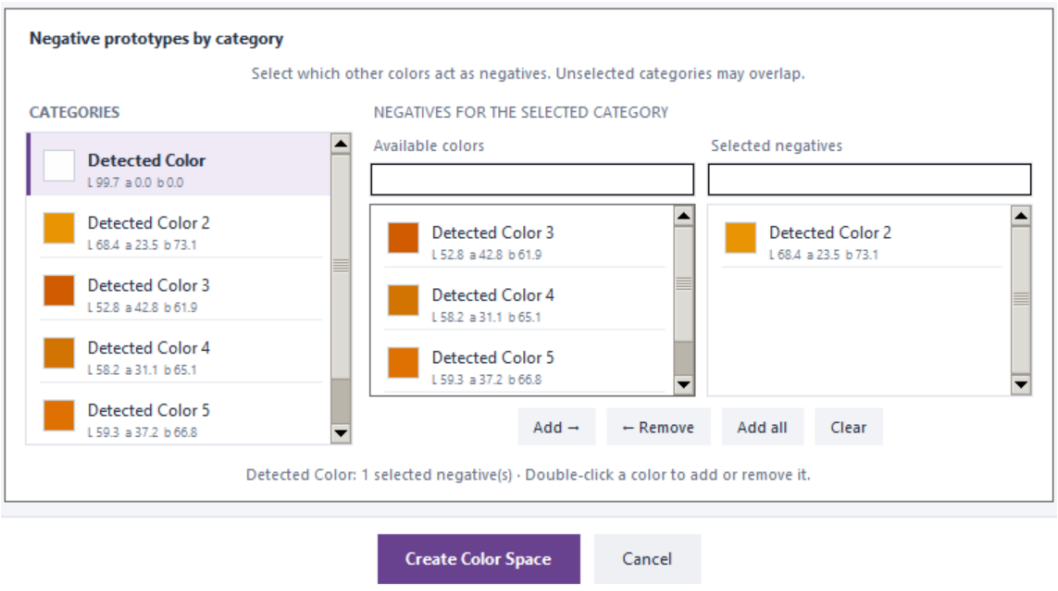


Figure 11: Selecting negative prototypes for an individual category in a non-disjoint FCS.

9.5 Auxiliary Sampling for Non-Covering Spaces

When `Covering` is unchecked, select the **Samples** setting: 8, 27, 64, 125, 216, 343, or 512. These settings specify the sampling density used to generate candidate auxiliary negatives near the reference domain. The number is a sampling-grid setting, *not* necessarily the number of negatives ultimately retained in the final geometry.

Higher densities may provide finer control of the uncovered regions but can increase construction time. For a covering space, the auxiliary sample setting is not used.

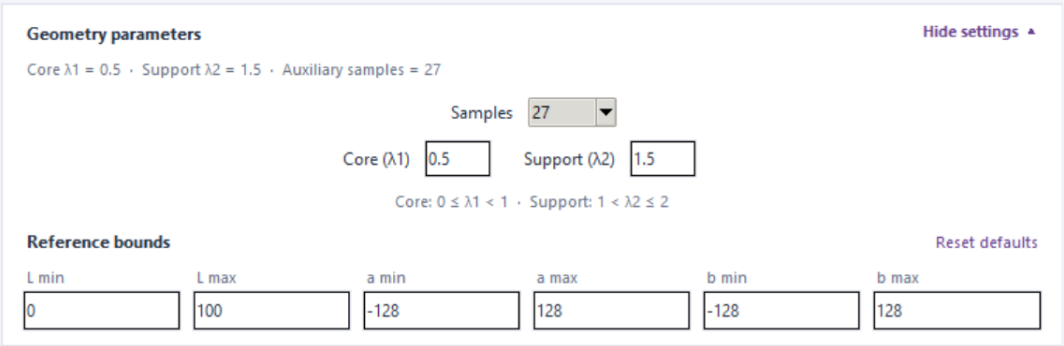


Figure 12: Auxiliary sampling options for a non-covering fuzzy color space.

9.6 Creating and Saving the Model

After configuring the options, enter a name and press `Create Color Space`. PyFCS validates the geometry parameters, reference bounds, and representatives. The **Processing** dialog indicates that geometric construction is underway; writing the completed `.fcs` may show a measured percentage.

If the name refers to an existing different `.fcs`, PyFCS requests confirmation before overwriting it. Declining this confirmation returns to the configuration dialog without closing the creation workspace.

The resulting `.fcs` stores the selected model and construction metadata alongside the prototypes and their geometric volumes.

10 Model 3D

The **Model 3D** tab visualizes the currently loaded fuzzy color space in its native geometric coordinate system: **CIELAB** (L^*, a^*, b^*) or **RGB** (R, G, B).

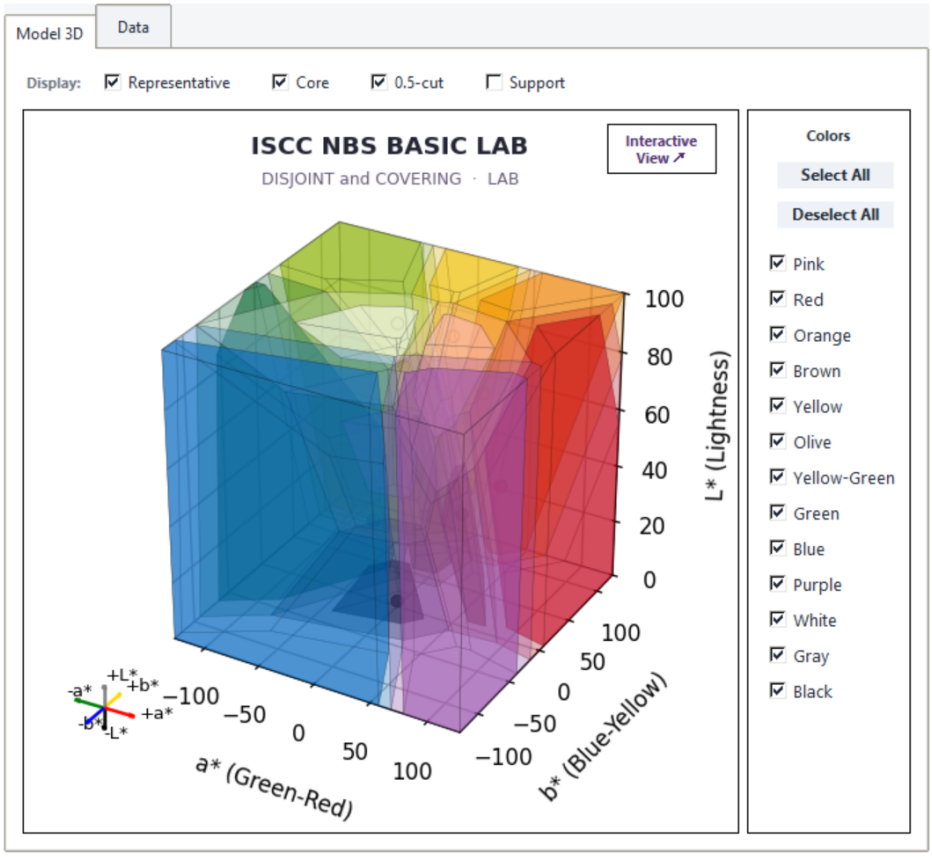


Figure 13: Model 3D tab showing a fuzzy color space in CIELAB coordinates.

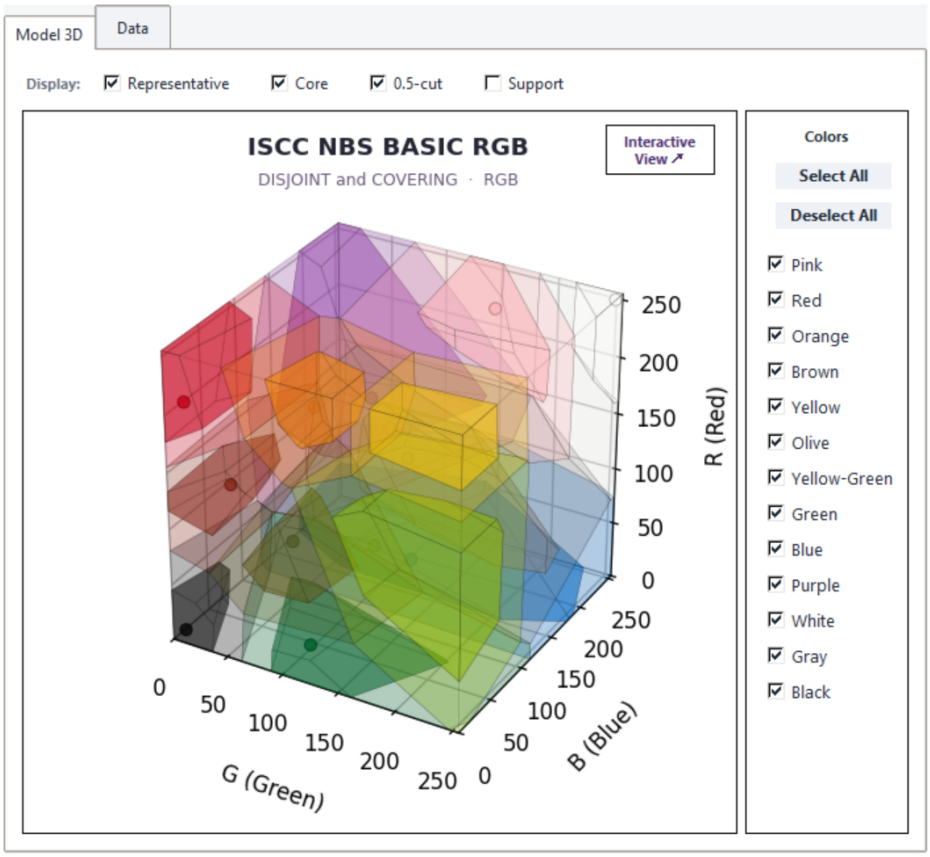


Figure 14: Model 3D displaying the Core, 0.5-cut, and Support in RGB coordinates.

10.1 Display Options

The following checkboxes control which geometric levels are drawn:

- `Representative`;
- `Core`;
- `0.5-cut`;
- `Support`.

Several levels may be enabled simultaneously.

The color list at the right side determines which fuzzy colors are included in the plot. Use `Select All` or `Deselect All` to change the whole selection efficiently.

10.2 Interactive Figure

`Interactive Figure` opens an external interactive representation of the current 3D configuration in the selected model coordinates. It is intended for free rotation, zooming, and closer spatial inspection.

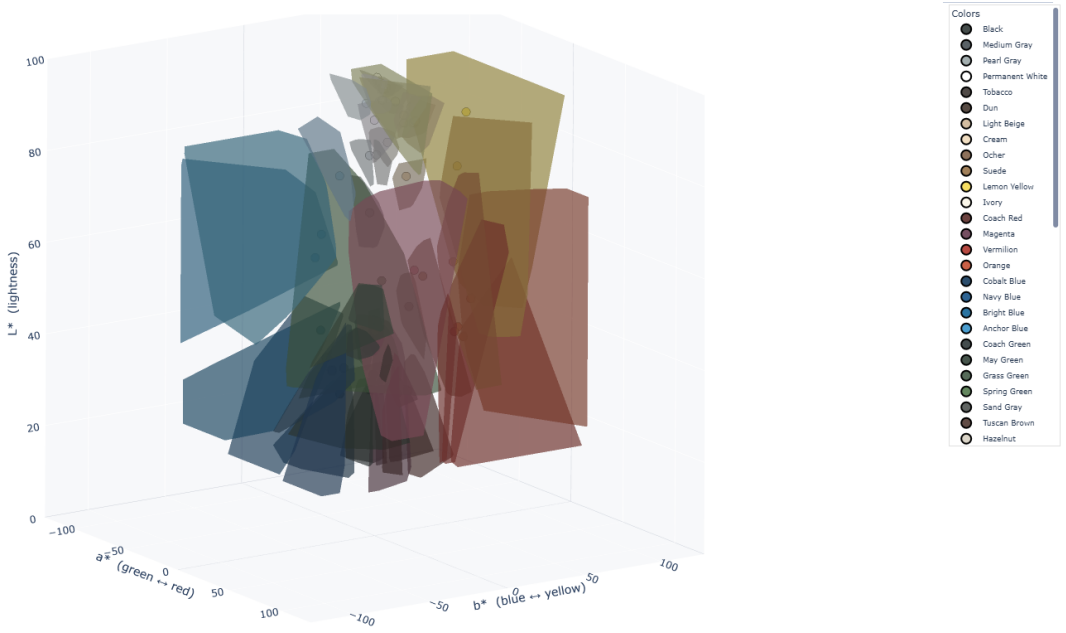


Figure 15: Interactive 3D visualization.

11 Data Tab

The **Data** tab provides an editable representation of the active color space.

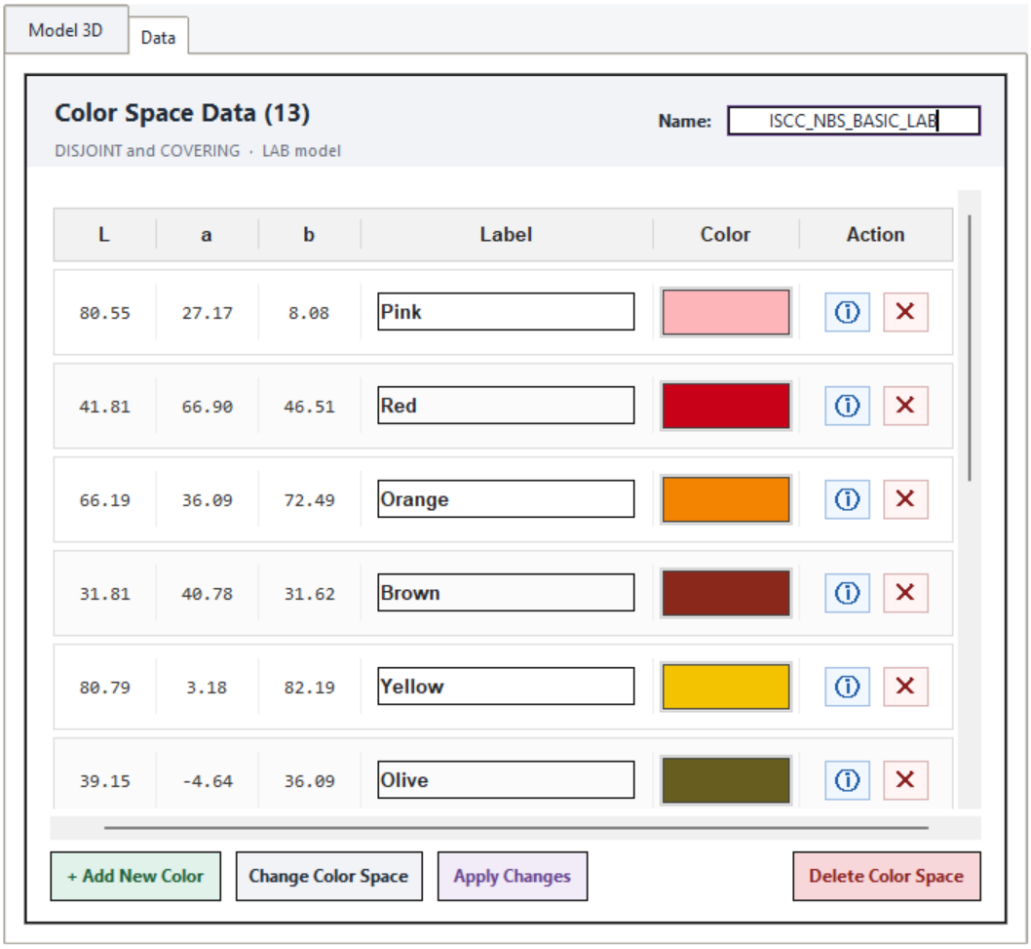


Figure 16: Data tab with editable labels and prototype information.

For each fuzzy color, the tab shows its color representative in the active model (LAB or RGB), editable label, color preview, and row actions. The current model and typology are displayed with the color-space name.

Control	Function
Editable **Name**	Rename the fuzzy color. Pending text is committed when `Apply Changes` is pressed, even if the user presses the button immediately after typing.
Information icon	Displays additional color information, including RGB and HEX information, for the corresponding representative.
Delete row action	Removes the color from the editable dataset. At least two colors must remain.
`+ Add New Color`	Adds a new representative through the reusable custom-color dialog.
`Change Color Space`	Open the dedicated RGB/CIELAB conversion dialog and rebuild the geometry in the selected coordinate system.
`Apply Changes`	Validate the current data, write the updated `.fcs`, rebuild geometry when necessary, invalidate incompatible mappings, and refresh the visualization.

Control	Function
`Delete Color Space`	Permanently deletes the currently loaded color-space file after confirmation and clears the corresponding application state.

Important. Changing a representative changes the fuzzy geometry. Existing image mappings belonging to the previous version of that color space must not be interpreted as mappings of the updated one. PyFCS invalidates the corresponding mapping state and recalculates new results from the original image when requested.

12 Changing Between RGB and CIELAB

The Data tab provides `Change Color Space` for converting an already loaded FCS between RGB and CIELAB geometry. This is an explicit *reconstruction* of the model, not a display toggle.

1. Load a valid FCS and open the **Data** tab.
2. Ensure that no color-mapping or other incompatible background task is running. Apply any pending Data edits first.
3. Press `Change Color Space`.
4. Choose `LAB` or `RGB` and press `Rebuild Geometry`.
5. Allow the progress indicator to complete. Review the updated 3D geometry and active model.
6. Use `Apply Changes` to save the rebuilt model to disk.

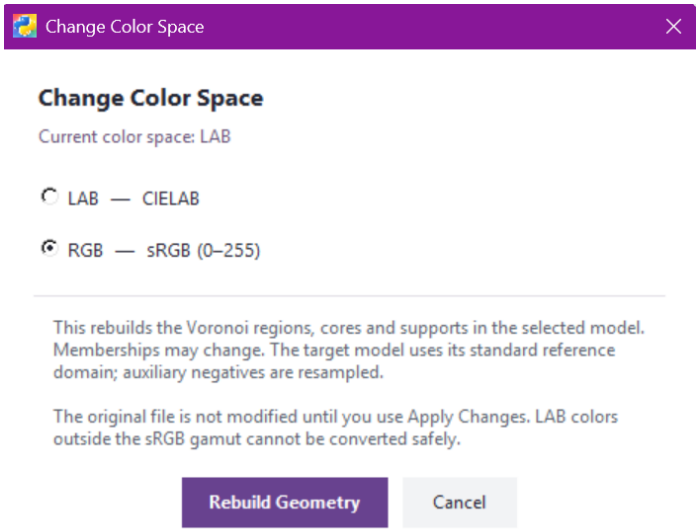


Figure 17: Change Color Space dialog for rebuilding a loaded model in RGB or CIELAB.

Important. Conversion changes the coordinate system in which Voronoi regions, cores, and supports are reconstructed; membership degrees can therefore change. The target model uses its standard reference domain, and auxiliary negative samples are regenerated where needed. Colors outside the sRGB gamut cannot always be converted safely from LAB to RGB.

The original `.fcs` is not modified by the conversion alone. If reconstruction fails, the previous valid model remains active. Do not assume that LAB and RGB geometries are mathematically identical simply because their representative colors look the same.

13 Image Manager and Image Display

The Image Manager controls the images used by the main application.

13.1 Opening Images

Press `Open Image` and select an image. Multiple images can remain open at the same time. Supported common formats include JPEG, PNG, BMP, GIF, TIFF, and WebP when supported by the local Pillow installation.

Every image is shown inside an **image card**. The card is the basic visual unit of the Image Display workspace.

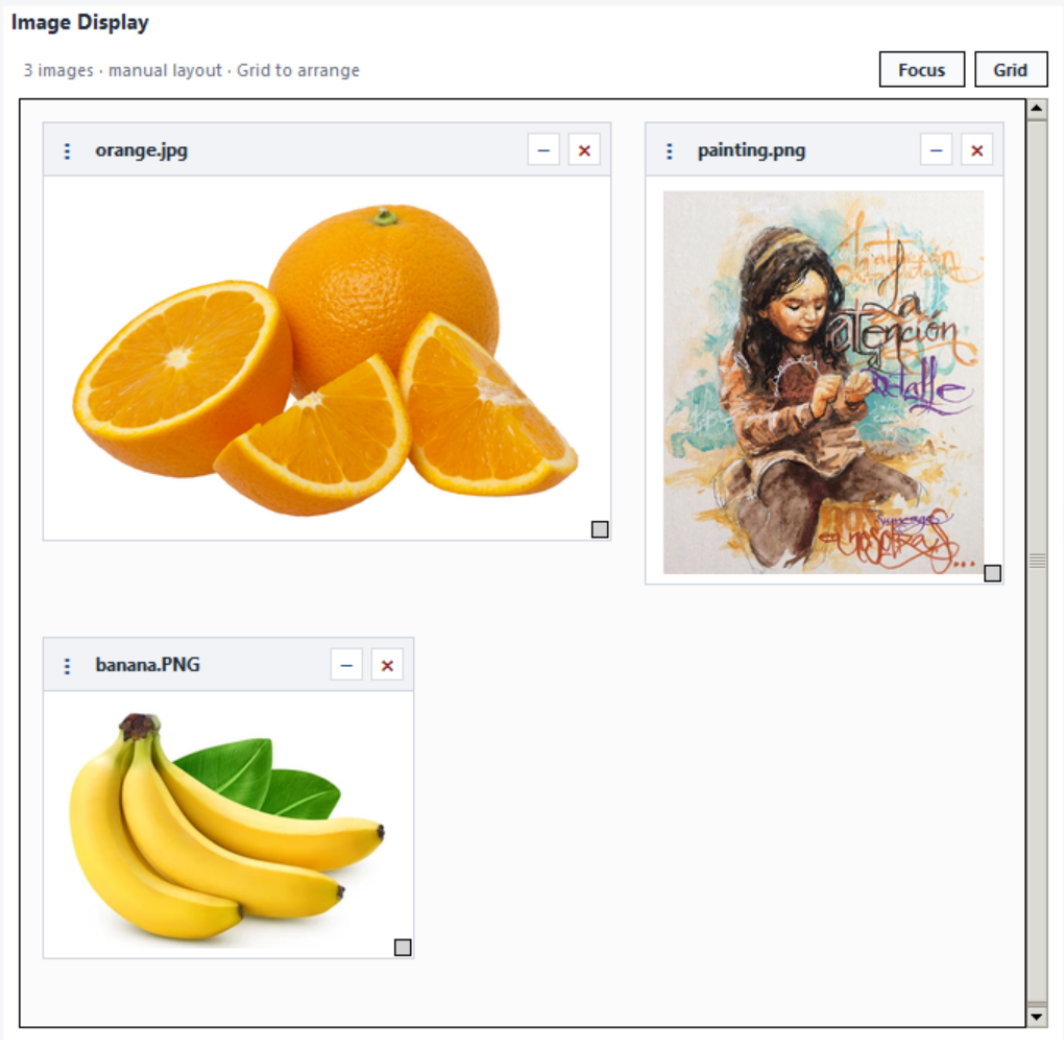


Figure 18: Image Display with several automatically arranged image cards.

13.2 Grid, Manual Layout, and Focus

The Image Display has three closely related layout states.

13.2.1 Grid

`Grid` automatically arranges all open cards without overlaps. The number of columns depends on the available width.

Use Grid when:

- several images must be compared at once;
- manual movement has produced overlaps;
- the main window has been resized significantly.

13.2.2 Manual Layout

Drag an image card by its title bar to move it freely. Resizing or moving a card switches the workspace to manual layout.

Pressing `Grid` at any time returns to the automatic arrangement.

13.2.3 Focus

Select an image card and press `Focus`, or choose `Focus Image` from its menu. The selected card expands to use the Image Display area.

While Focus is active:

- the global `Focus` button is disabled;
- the per-image Focus command is disabled;
- manual resizing is disabled;
- `Grid` is the normal way to leave Focus mode.

Selections are cleared when the geometry changes between Grid and Focus so that an old ROI cannot be visually associated with a new image geometry.



Figure 19: Focus mode for a single image.

13.3 Image Card Controls

Control	Function
⋮	Open the per-image menu.
Title bar	Left-drag to move the complete card in manual layout.
−	Minimize the card to its header. When minimized, the control changes to + to restore it. It is hidden in Focus mode.
×	Close the image and remove its window-specific state from the workspace.
Bottom-right resize handle	Resize the image card. Resizing switches the workspace to manual layout. The handle is disabled in Focus mode.
Mouse wheel over image	Zoom around the cursor.
Right-drag or middle-drag over image	Move the internal viewport when zoomed. This moves the image content, not the card itself.

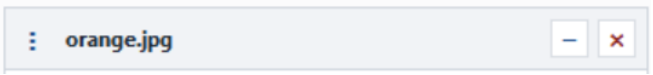


Figure 20: Image card controls.

13.4 Per-Image Menu

The ⋮ menu contains:

Menu item	Function
`Focus Image`	Enter Focus mode for this image.
`Original Image`	Return to the original image view.
`Color Mapping`	Open the single-prototype mapping selector.
`Color Mapping All`	Map every valid pixel to its winning fuzzy prototype.
`Zoom → Zoom In`	Increase zoom.
`Zoom → Zoom Out`	Decrease zoom.
`Zoom → Fit`	Reset the viewport and fit the complete image in its card.
`Save Image`	Save this card directly without first opening the global image selector.

The three view modes—Original Image, Color Mapping, and Color Mapping All—can be selected directly from one another. It is not necessary to return manually to Original Image between mappings. New calculations always use the stored original image as their processing source.

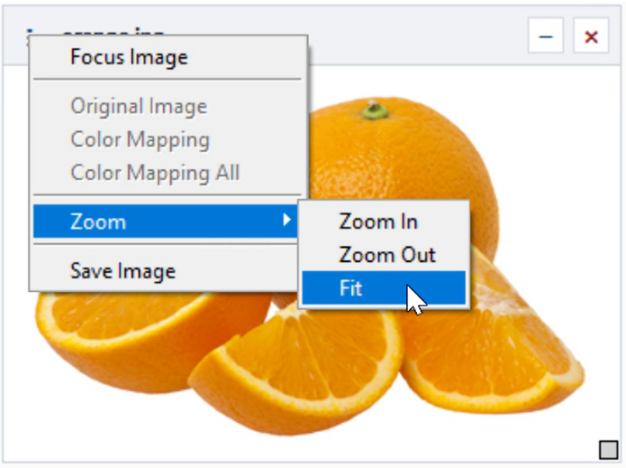


Figure 21: Per-image menu.

13.5 Pixel and ROI Sampling in the Main Display

When original-image sampling is active:

- a click selects one original-image pixel;
- a drag selects a rectangular ROI;
- a very small drag is treated as a pixel click;
- ROI color is computed from the original image pixels.

The coordinate conversion is viewport-aware. Zoom, Grid, Focus, image-card resizing, and Canvas scrolling do not change which original pixel corresponds to the cursor.

For deeper analysis, use **Image Inspection**, described later in this manual, Section 16.

14 Color Mapping

Color Mapping requires an active fuzzy color space.

14.1 Single-Prototype Color Mapping

Choose `Color Mapping` from an image card.

A compact prototype panel appears below the image. Initially no prototype is selected. Selecting one fuzzy color computes and displays its membership map.

The panel shows the available prototypes as radio options, the image card shows the percentage associated with the selected mapping, and the `Grayscale/Color` control offers two views of the same membership map:

- `Grayscale`: show membership intensity as a grayscale map;
- `Color`: weight the original image colors by the membership degree to retain chromatic context.

Switching the display style does not recompute memberships or change the reported percentage.

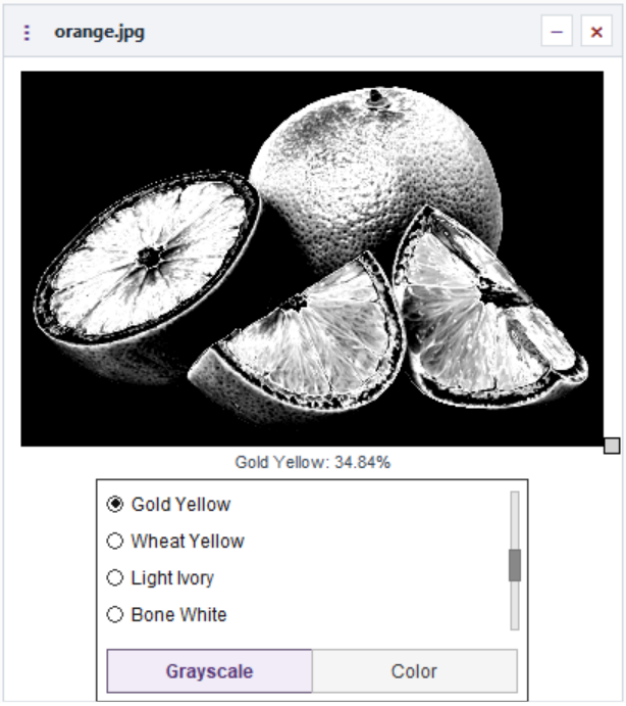


Figure 22: Single-prototype Color Mapping with the prototype selector below the image.

The calculation always starts from the original image, even if another mapping or a zoomed view was previously visible.

14.2 Color Mapping All

Choose `Color Mapping All` to classify every valid pixel according to the fuzzy prototype with the highest membership.

The result is a recolored image accompanied by a compact vertical legend below the image card.



Figure 23: Color Mapping All with its integrated legend.

The panel offers `Original` and `Alt. Colors` visualization palettes. `Original` uses representative colors; `Alt. Colors` uses an alternative palette for easier visual separation. Switching between the two does not recompute fuzzy memberships or change the label map.

> **Tip.** Color Mapping All is useful for interpreting the image as a linguistic fuzzy shade map. The displayed recoloring is a visualization of the winning prototype, not a replacement of the stored original image.

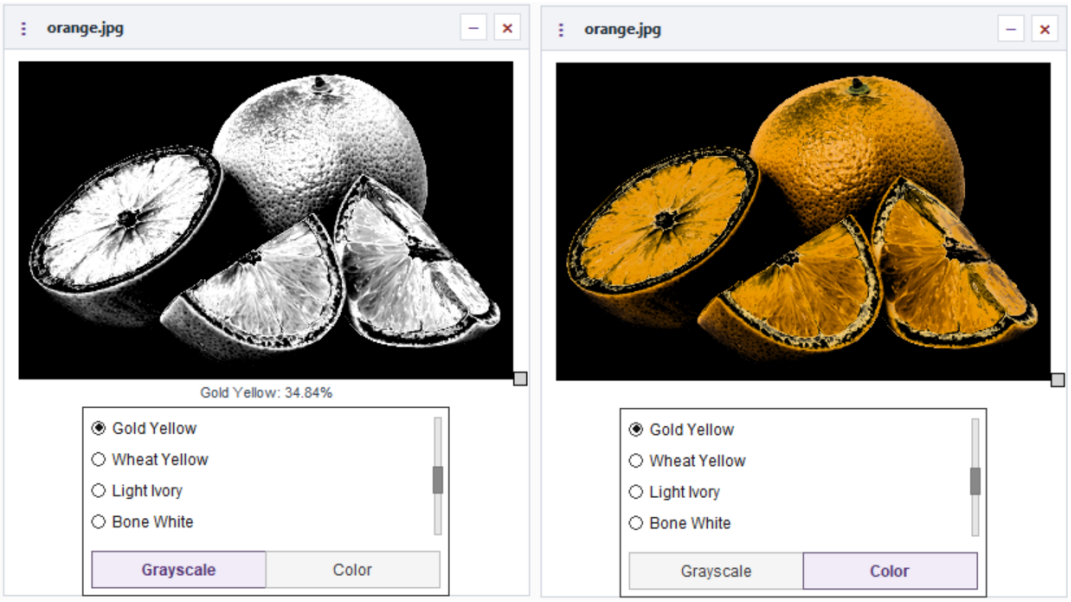


Figure 24: Comparison of Grayscale and Color views for one fuzzy membership map.

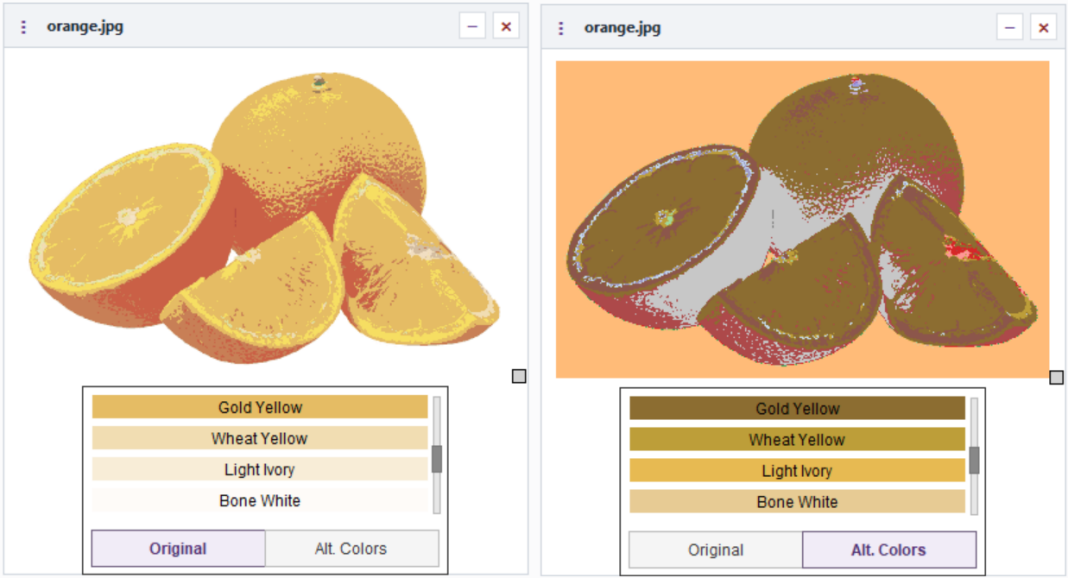


Figure 25: Original and alternative palettes for the same Color Mapping All result.

15 Saving Images

There are two ways to save an image.

15.1 Saving Directly from an Image Card

Open the card menu and choose `Save Image`. PyFCS saves the selected card directly.

The exported image corresponds to the **complete current view source** (Original, Color Mapping, or Color Mapping All). Zoom and panning are viewport operations and therefore do not crop the exported result.

15.2 Global Save Image Selector

Press the main toolbar `Save Image`. A visual selector shows the open images using:

- a thumbnail of the current view;
- the image file name;
- the current mode;
- an indication when Mapping All will also export a legend.

Select a row and press `Save Selected`. Double-clicking a row also starts the save action.

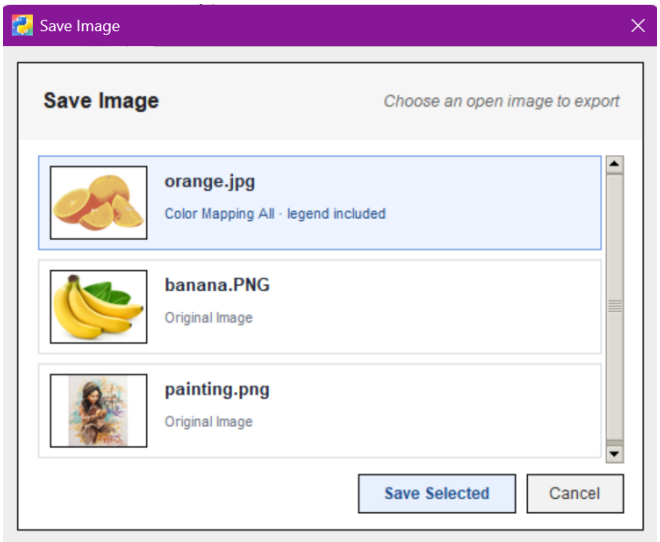


Figure 26: Global Save Image selector.

15.3 Mapping-All Legend Export

When the saved image is in Color Mapping All mode, PyFCS also writes a standalone PNG legend using the palette currently visible in the card.

For example:

```
result.png
result_legend.png
```

If `Alt. Colors` is active, the exported legend corresponds to that active palette.

16 Image Inspection

Image Inspection is the dedicated workspace for detailed analysis of image colors against the currently loaded fuzzy color space.

Open it with the toolbar `Image Inspection` or from the Image menu.

A fuzzy color space is required. An image is not required before opening the workspace because images can be selected or opened from within it.

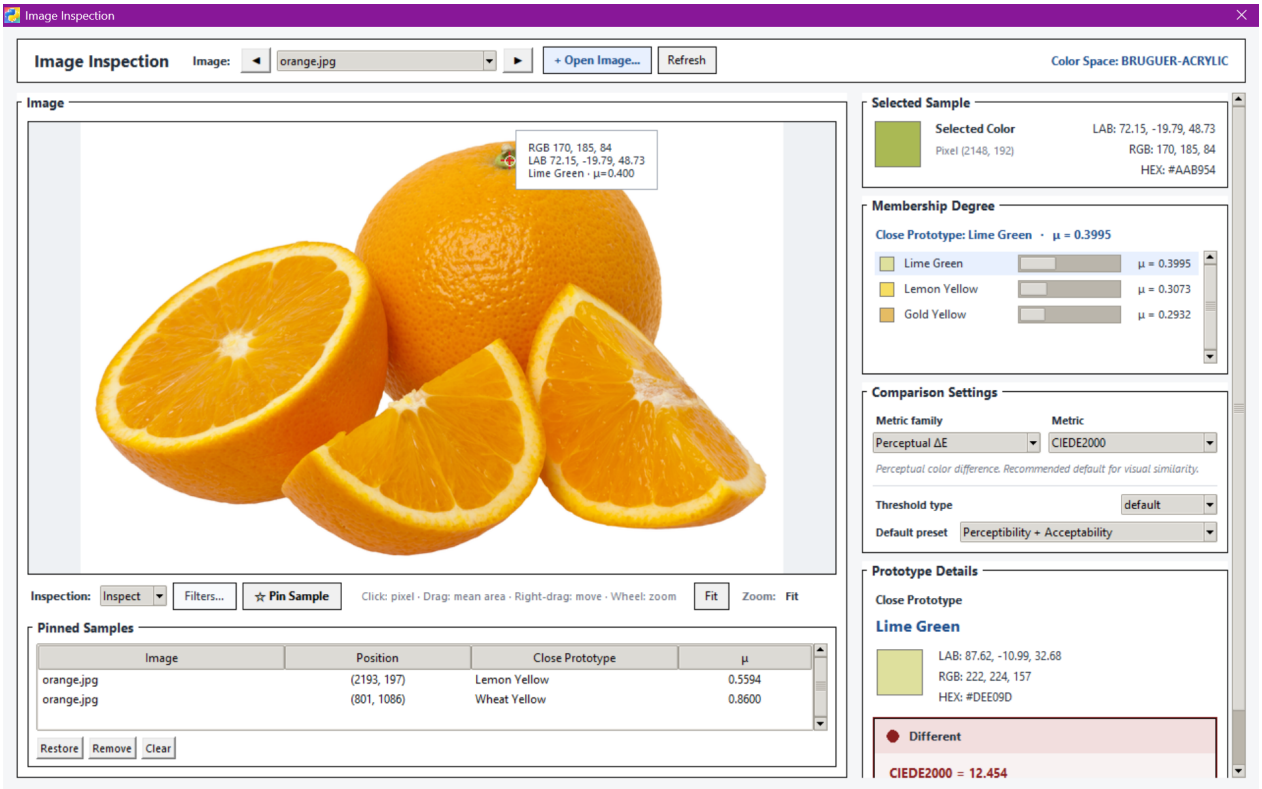


Figure 27: Image Inspection workspace.

16.1 Image Source Controls

Control	Function
◁ / ▷	Move to the previous or next available image.
Image selector	Choose a source directly. The workspace can contain main-workspace images and locally opened temporary images.
`+ Open Image...`	Open an image locally inside Image Inspection without adding it to the main Image Display.
`Refresh`	Refresh the list of images coming from the main Image Display.
`Fit`	Reset the image to a fitted view.
Mouse wheel	Zoom around the cursor.
Right-drag / middle-drag	Move the zoomed viewport.

16.2 Inspection Modes

The `Inspection` selector provides two modes.

Inspect. Use a normal click to inspect a pixel. Drag a rectangle to inspect the mean color of an ROI.

Live. Move the pointer over the image to update the inspected pixel interactively. This is useful for quickly exploring membership changes without repeatedly clicking.

Changing mode cancels an unfinished manual ROI selection.

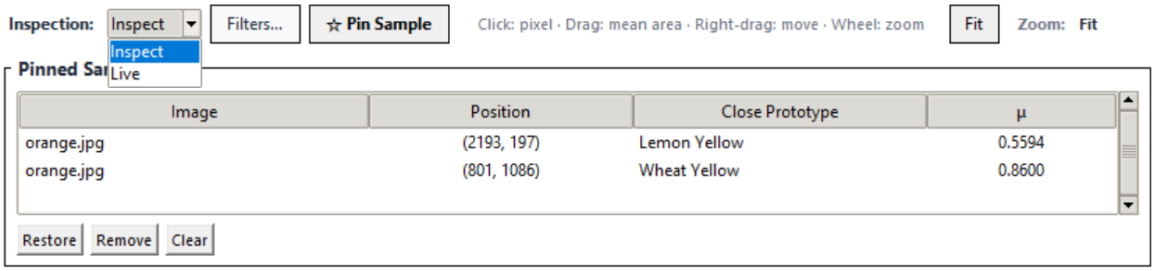


Figure 28: Inspect and Live interaction modes.

16.3 Selected Sample

The **Selected Sample** card reports:

- sample type and image position;
- RGB;
- CIELAB;
- HEX;
- color preview.

For an ROI, the displayed color represents the ROI mean.

16.4 Membership Degree

The **Membership Degree** section computes the fuzzy memberships of the sample against the active FCS.

The row with the highest membership is summarized as the **Close Prototype**. Other non-zero memberships remain visible, allowing the user to inspect ambiguity and overlap between fuzzy colors.

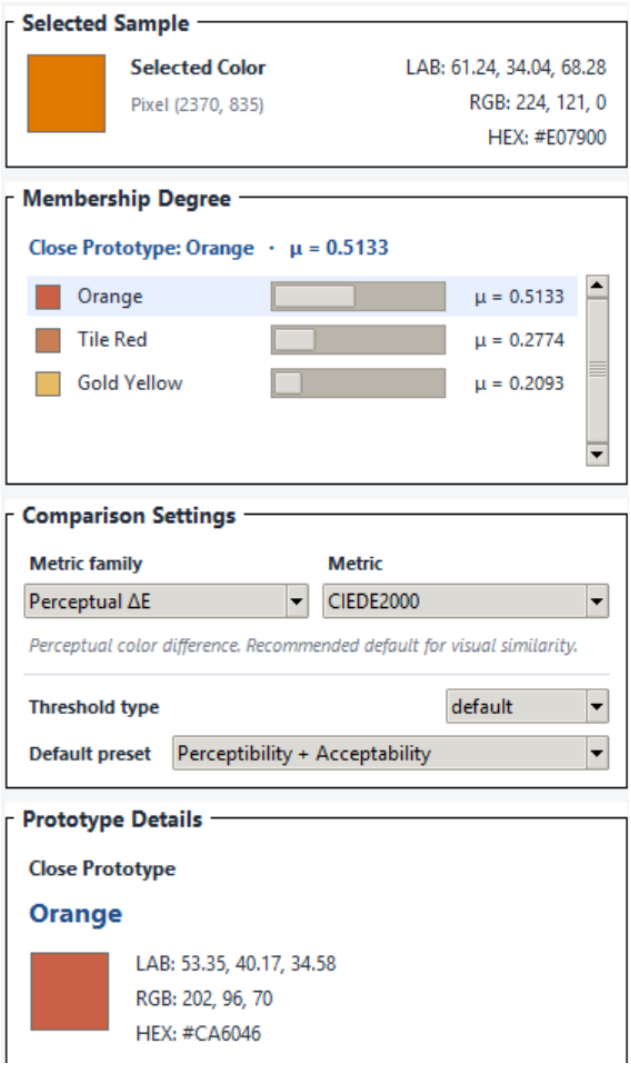


Figure 29: Membership degrees and Close Prototype in Image Inspection.

16.5 Comparison Settings

Image Inspection reuses the same configurable color-difference system as Color Evaluation.

The main controls are:

- **Metric family**;
- **Metric**;
- **Threshold type**: default or custom;
- **Default preset**, when the default mode is active;
- custom single or lower/upper threshold values, when custom mode is active.

The comparison is performed between the inspected sample and the selected prototype, with the Close Prototype used by default.

16.6 Prototype Details

The **Prototype Details** section shows the selected fuzzy prototype, including:

- its label;
- CIELAB representative;
- RGB;
- HEX;
- color preview;
- the active color-difference result and threshold interpretation.

16.7 Pinned Samples

Press ⋆ `Pin Sample` to preserve the current sample in the **Pinned Samples** table.

The table stores:

- image;
- position;
- Close Prototype;
- membership degree μ.

Available actions are:

Control	Function
`Restore`	Return to the selected pinned sample. Double-clicking its row provides the same action.
`Remove`	Remove only the selected pinned sample.
`Clear`	Remove all pinned samples from the inspection session.

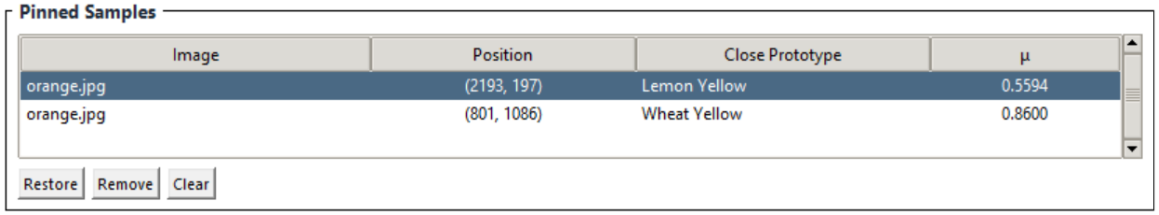


Figure 30: Pinned Samples in Image Inspection.

17 Color Evaluation

The `Color Evaluation` workspace provides a dedicated environment for quantitative and visual comparison of colors.

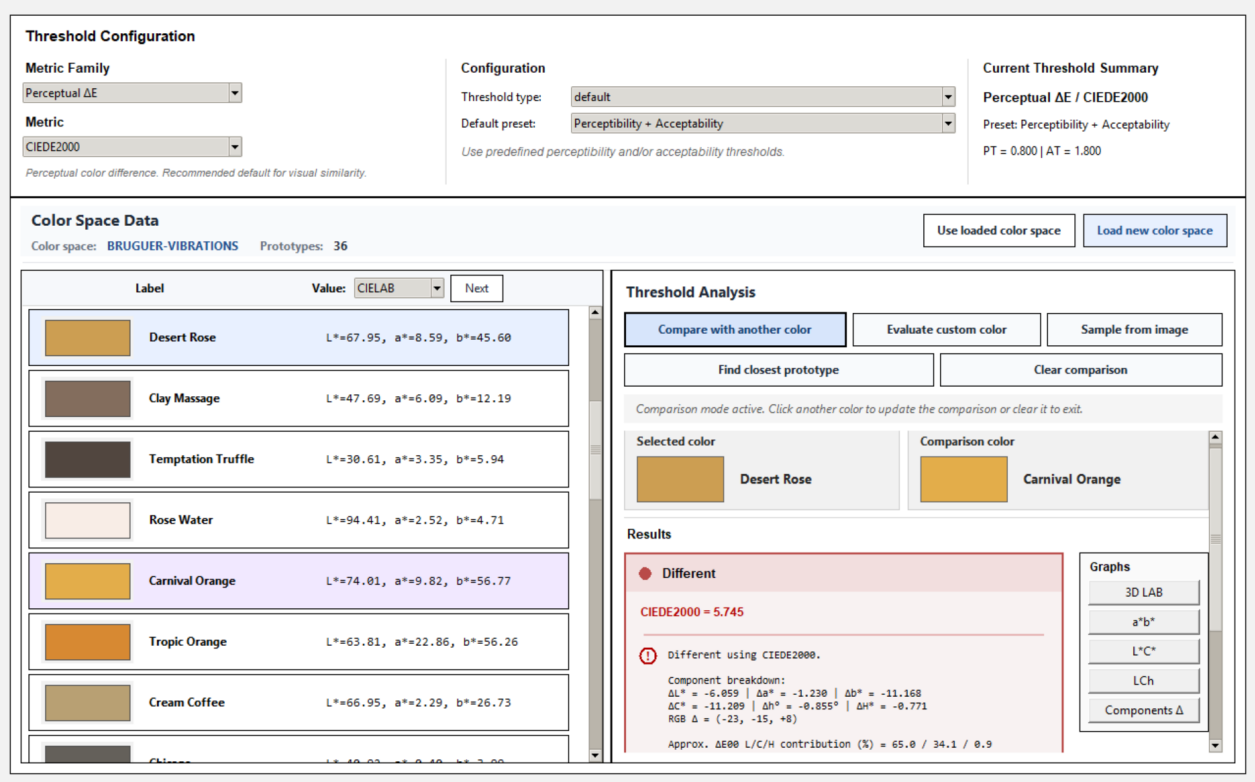


Figure 31: Color Evaluation workspace.

17.1 Color Space Data

The workspace can use the color space already loaded in the main application or load another source for the evaluation session.

The first stage presents the available prototypes and allows one to be chosen as the reference color.

17.2 Comparison Sources

The comparison panel supports several workflows:

- `Compare with another color`: compare prototypes;
- `Evaluate custom color`: enter a custom comparison color;
- `Sample from image`: sample a pixel or ROI from an image and use it as the comparison color;
- `Find closest prototype`: rank the closest prototypes for the active custom/image sample;
- `Clear comparison`: reset the current comparison state.

17.3 Metrics and Thresholds

The default perceptual metric is CIEDE2000. The evaluation system also supports other perceptual and component-based color-difference families exposed by the Metric family and Metric controls. CIELAB-based color differences are calculated using the corresponding color values even when the loaded FCS geometry was constructed in RGB; a perceptual difference metric should not be confused with the space used to generate the volumes.

Thresholds can be used in:

- **default mode**, using the configured PT/AT-style presets;
- **custom mode**, using either one threshold or a lower/upper interval.

The result card summarizes the numerical value and its interpretation using the active threshold configuration.

17.4 Closest Prototypes and Membership Analysis

For a custom color or image sample, `Find closest prototype` ranks the seven closest available prototypes using the active metric.

Membership information can be inspected alongside distance-based comparison, providing both fuzzy-category and color-difference perspectives.

This makes it possible to compare the nearest prototypes not only according to their numerical color difference, but also according to the fuzzy membership degrees of the evaluated color.

17.5 Graphical Analysis

The workspace also provides graphical views for interpreting the relationship between the selected and comparison colors.

Available visualizations include:

- a **3D CIELAB representation**, showing the position of the evaluated colors in the color space;
- a **2D a^*b^* representation**, providing a direct view of their chromatic separation;
- **membership graphs**, showing the fuzzy membership degrees associated with the evaluated color.

These graphical representations complement the numerical color-difference results and make it easier to interpret both geometric distance and fuzzy membership relationships.

18 Display Threshold

The toolbar `Display Threshold` contains two visualization actions:

- **Perceptibility Threshold (PT)**: default threshold $\Delta E = 0.8$;
- **Acceptability Threshold (AT)**: default threshold $\Delta E = 1.8$.

These actions visualize the connected set of sampled points satisfying the chosen threshold around the selected representative. The exploration uses a discrete sampling grid; it does not guarantee that a separate, spatially disconnected valid component elsewhere will also be detected. The search no longer stops after an arbitrary number of consecutive unsuccessful sample points.

19 Recommended Workflows

19.1 Workflow A: Build an FCS from a Known Palette

1. Open `Palette-Based Creation`.

2. Select colors from the default source.
3. Optionally use `+ Add Color Source...` to accumulate reference colors from additional CNS/FCS files.
4. Rename colors locally when needed.
5. Add custom colors if required.
6. Select the final prototypes.
7. Press `Create Color Space`.
8. Resolve any duplicate-prototype warning before saving.
9. In Color Space Configuration, select LAB/RGB and the desired Disjoint/Covering typology.
10. Adjust geometry parameters, reference bounds, and any negative prototypes or auxiliary sampling required by the chosen typology.
11. Name the new FCS and optionally load it immediately.

19.2 Workflow B: Build an FCS from an Image Manually

1. Open `Image-Based Creation`.
2. Select an existing image or use `+ Open Image...`.
3. Zoom and move to the region of interest.
4. Click individual representative pixels or drag ROIs.
5. If necessary, configure LAB Filters before taking ROI means.
6. Add each useful sample to Selected Colors and rename it.
7. Add custom colors if required.
8. Select the final set and press `Create Color Space`.
9. Configure the FCS model, typology, and geometric parameters before saving.

19.3 Workflow C: Automatic Image Palette Extraction

1. Open `Image-Based Creation`.
2. Select an image.
3. Configure the automatic detection settings.
4. Optionally enable LAB Filters.
5. Press `Detect and Add Colors`.
6. Review, rename, select, or remove detected colors.
7. Repeat on other images if the FCS should represent several sources.
8. Configure the geometry and create the final FCS.

19.4 Workflow D: Map an Image with an Existing FCS

1. Load the FCS.
2. Open one or more images.
3. Use Grid for comparison or Focus for detailed viewing.
4. Open an image menu.
5. Choose `Color Mapping` to study one prototype, or `Color Mapping All` for complete fuzzy labeling.
6. Zoom and right-drag when closer visual inspection is needed.
7. Use `Save Image` to export the complete result.
8. For Mapping All, keep the automatically generated `_legend.png` file with the image.

19.5 Workflow E: Inspect a Sample in Detail

1. Load the target FCS.
2. Open `Image Inspection`.
3. Choose or open an image.
4. Use Inspect mode for a fixed pixel/ROI or Live mode for exploration.
5. Review the sample RGB/LAB/HEX values.
6. Inspect the membership distribution and Close Prototype.
7. Configure the desired metric and threshold.
8. Pin important samples for later restoration and comparison.

19.6 Workflow F: Rebuild an Existing FCS in Another Model

1. Load a valid LAB or RGB FCS.
2. Open the Data tab and apply any pending changes.
3. Choose `Change Color Space`.
4. Select the target model and press `Rebuild Geometry`.
5. Review the reconstructed 3D model and representative coordinates.
6. Press `Apply Changes` when ready to persist the result.

20 File Formats and Output Behavior

20.1 Color-Space Files

Format	Use in PyFCS
`.cns`	Palette/color-name source. It can be loaded by the main FCS manager and can also be imported as a color source in Palette-Based Creation.
`.fcs`	Complete fuzzy color-space representation. The same extension supports legacy files and extended metadata for RGB/CIELAB, typology, scaling, reference bounds, negatives, and forward-compatible granularity fields. It can be loaded for analysis or used as a representative-color source.

20.2 Compatibility and Granularity

PyFCS 1.5.0 continues to use `.fcs` for both older and extended model files. No separate `.gfcs` format is required. The current application constructs **atomic fuzzy colors**, with one positive prototype per category. The extended serializer can represent granule counts and t-conorm metadata for future models, but support for editing and constructing multiple positive sub-prototypes per color is not yet part of this release.

The importer handles legacy and extended supported representations; older versions of third-party tools may not understand newly written metadata. Keep a backup when exchanging files with older software.

20.3 Image Outputs

When saving an Original or single Color Mapping result, one image file is written.

When saving a Color Mapping All result, PyFCS writes:

- the mapped image at the selected path;
- a separate PNG legend with the same base name and suffix `_legend`.

21 Mouse and Keyboard Reference

21.1 Global Keyboard Controls

Input	Action
`F11`	Toggle fullscreen/windowed mode in the main application.
`Escape`	Toggle fullscreen/windowed mode in the main application. In modal dialogs, Escape commonly cancels or closes the active dialog where implemented.
`Alt+F`	Open File menu on Windows/Linux.
`Alt+I`	Open Image menu on Windows/Linux.
`Alt+C`	Open Fuzzy Color Space menu on Windows/Linux.
`Alt+A`	Open Color Analysis menu on Windows/Linux.
`Alt+H`	Open Help menu on Windows/Linux.

21.2 Image Interaction Reference

Input	Action
Mouse wheel	Zoom image around the cursor.
Right-drag / middle-drag	Move the zoomed image viewport.
Left-drag on card title	Move an image card in the main Image Display.
Left-drag resize handle	Resize a main Image Display card.
Left click on original image	Select/sample a pixel when sampling is active.
Left drag on original image	Select an ROI and compute its mean when sampling is active.

22 Good Practice

- Use **Grid** whenever multiple cards become difficult to compare after manual movement.
- Use **Focus** for detailed work rather than repeatedly enlarging a card manually.
- Treat zoom and viewport movement as visualization tools. Saving exports the complete current image result rather than only the visible crop.
- For ROI-derived color representatives, use LAB Filters in Image-Based Creation when highlights, shadows, or unwanted chromatic regions would bias the mean.
- Avoid coincident prototypes in the chosen geometric model. PyFCS blocks them because they are not valid distinct Voronoi sites.
- Choose LAB or RGB geometry deliberately: the same displayed colors may produce different volume shapes and memberships.
- For non-disjoint models, review the negative-prototype list of each category; for non-covering models, check sampling density.
- Review the reference bounds and core/support scaling before starting a large geometric construction.
- When the FCS is edited or replaced, allow PyFCS to regenerate mappings from the original image rather than interpreting an old mapping with the new space.
- Preserve the `_legend.png` output together with Mapping-All figures when results will be included in a paper, report, or dataset.

23 Troubleshooting

23.1 A Mapping Option Is Disabled

Verify that a valid fuzzy color space is loaded. Mapping actions depend on the active FCS.

If an old Mapping-All result remains visible after changing the FCS, request a new mapping. PyFCS will calculate it from the original image and the current color space.

23.2 An Image Card Overlaps Another

Press `Grid`. Moving or manually resizing cards intentionally switches the workspace to manual layout.

23.3 Focus Cannot Be Pressed Again

This is intentional. Focus is an exclusive mode. Use `Grid` to leave it before focusing another card.

23.4 Right-Drag Does Not Move the Image

Viewport movement is only meaningful after zooming beyond Fit. Use the mouse wheel or the Zoom menu first.

23.5 The Wrong Pixel Appears to Be Selected

Use `Fit` and test again. The current implementation maps cursor coordinates through the active zoom viewport back to original-image coordinates. Grid/Focus changes also clear stale selections.

23.6 A New FCS Cannot Be Created

Check the following:

- at least two colors are selected;
- each representative is valid in the chosen model and domain;
- no two selected colors collide within the duplicate-prototype tolerance;
- core and support scaling satisfy the permitted intervals;
- disjoint models also satisfy $1 \leq \lambda_1 + \lambda_2 \leq 2$;
- the selected auxiliary sample density is valid for non-covering models;
- every required color has a valid name.

23.7 Change Color Space Does Not Complete

Finish or cancel active mapping operations first and apply pending Data edits. A conversion from LAB to RGB may be rejected if representative colors lie outside the sRGB gamut or if the rebuilt geometry is invalid. The existing model remains available if conversion is not applied.

23.8 Loading or Processing Appears Busy

Large geometries, non-covering auxiliary sampling, and full-image mapping can require additional processing. Loading and geometry reconstruction show animated activity rather than inventing a percentage when the total work is not measurable. Wait for the current operation to complete or use the available cancellation control where provided.

23.9 Mapping-All Legend Is Missing on Export

Confirm that the image is actually in Color Mapping All mode when it is saved. The legend is generated for Mapping-All exports and uses the palette currently displayed.

24 Contact and Support

For technical support:

rafaconejo@ugr.es

Project resources:

https://github.com/RafaelConejo/PyFCS
https://pyfcs.com/